Solving N-Queens with SAT: An Experimental Comparison of Cardinality Encodings and Exact Solvers

Do Duc Long[a,*]

[a]VNU University of Engineering and Technology, Hanoi, Vietnam

ARTICLE INFO

Keywords:
N-queens
SAT encoding
cardinality constraints
constraint programming
integer linear programming

ABSTRACT

This report implements an exact SAT solver for the standard N-queens puzzle and compares five at-most-one encodings: pairwise, binary, commander, recursive product, and sequential counter. A minimal formulation and a formulation with redundant column clauses are evaluated using PySAT. Four SAT backends are compared using a fixed encoding, followed by comparisons with IBM CPLEX CP Optimizer, IBM CPLEX MIP, Gurobi, and OR-Tools CP-SAT. The implementation uses the same problem semantics for every method, independently validates returned placements, and stores the raw solutions and experiment configurations. Experiments use fresh processes, one solver thread, three repetitions, and an enforced ten-second process deadline. Boards up to 512×512 are included to examine scaling. Results distinguish model construction and search from process startup and license acquisition, and account explicitly for timeouts. Commander/minimal achieves the lowest encoding PAR-2 (0.450 s). At $n = 512$, CPLEX CP and both MIP baselines succeed in all repetitions, while reference SAT and CP-SAT reach the deadline. The study concerns finding one solution to the unconstrained puzzle; it does not measure counting or the NP-complete completion problem.

1. Introduction

The N-queens puzzle asks for n queens on an $n \times n$ chessboard such that no two queens attack each other. It provides a compact example of exact constraint solving: every row must contain a queen, while columns and diagonals must exclude conflicts. A Boolean placement model turns these requirements into cardinality constraints. Its runtime can depend on how those constraints are translated into conjunctive normal form (CNF), even when the resulting formulas have exactly the same board solutions.

PySAT supplies a common interface to SAT engines and cardinality encoders [13]. This makes it possible to compare encoding choices without changing the application semantics. Constraint programming (CP) instead represents a queen by its column and expresses non-attacking conditions with global all-different constraints [9]. Integer linear programming (ILP) represents occupied squares with binary variables and linear equalities or inequalities. These alternative formulations allow a comparison between SAT and established exact solvers.

The present work contributes (i) an executable SAT solver with five AMO encodings and two formulations, (ii) a semantic verification suite that checks complete projected solution sets on small boards, (iii) comparisons with four SAT backends and all four CP/ILP solvers required by the course assignment, and (iv) reproducible measurements, raw placements, figures, and this report.

The standard puzzle has a solution for $n = 1$ and every $n \geq 4$, while $n = 2$ and $n = 3$ are infeasible. Consequently, deciding existence in this particular family should not be called NP-complete. The N-queens completion problem, where some queens are fixed in advance, is a different problem and is NP-complete [8]. Here the aim is to study the behavior of generic exact methods on the standard puzzle, rather than propose a new algorithm that supersedes explicit constructions.

2. Background and related work

2.1. SAT, cardinality constraints, and the compared models

SAT asks whether Boolean variables can be assigned so that a formula is true. In CNF, the formula is a conjunction of clauses, and each clause is a disjunction of literals. A literal is a variable or its negation. For example, $(\neg x_1 \vee \neg x_2)$

*Corresponding author
Email address: 24021550@vnu.edu.vn (D.D. Long)

excludes the assignment in which both inputs are true. If x_1 is already true, the remaining literal forces x_2 to be false. This inference is called unit propagation: a clause with only one unassigned literal and all other literals false forces that literal to be true. Repeating this rule can detect conflicts without making another search decision.

For a set of Boolean inputs, ALO requires at least one true input, AMO allows at most one, and EO requires exactly one. ALO needs a single disjunction, whereas AMO has several possible CNF representations. Extra variables can reduce the number of clauses but also change the paths through which propagation occurs. Thus two encodings with the same solutions need not take the same time to build or solve.

The models compared here express the same feasibility task using different variables. SAT and ILP use one occupancy variable per square; the CP model uses one integer column variable per row. SAT receives clauses, ILP receives linear equalities and inequalities, and CP receives three global all-different constraints. These user models are described explicitly in the following section; their sizes do not describe the internal models after solver preprocessing.

2.2. Related work

N-queens. Bell and Stevens [2] survey the extensive literature on the puzzle, including explicit constructions that place n non-attacking queens for every $n \geq 4$ in polynomial time. Finding one placement on an empty board is therefore not a hard decision problem, but the puzzle remains a common benchmark for constraint models because its row, column, and diagonal constraints are easy to state and grow quickly with n. Gent et al. [8] proved that the completion variant, in which some queens are fixed in advance, is NP-complete and that counting its solutions is #P-complete.

At-most-one encodings. Prestwich [20] reviews CNF encodings of finite-domain and cardinality constraints. The pairwise encoding needs no auxiliary variables but a quadratic number of clauses. The binary (bitwise) encoding of Frisch and Peugniez [7] uses a logarithmic number of auxiliaries. Sinz [23] introduced the sequential counter, whose linear-size AMO case maintains arc consistency through unit propagation. Klieber and Kwon [14] proposed the commander encoding, which applies AMO hierarchically to groups, and Chen [4] the product encoding, which projects inputs onto a two-dimensional grid. Nguyen and Mai [18] combined the binary and commander ideas in the bimander encoding. Comparative studies such as Frisch and Giannaros [6] report that no encoding is best across all benchmarks, because formula size and propagation strength pull in different directions. This motivates measuring the encodings on the specific problem rather than choosing by clause count.

Exact solvers. Modern SAT solvers follow the conflict-driven clause learning (CDCL) scheme introduced by GRASP [16]. The backends evaluated here are MiniSat [5], Glucose, which ranks learned clauses by literal block distance [1], CaDiCaL [3], and MapleChrono, which adds chronological backtracking [17]; all are accessed through PySAT [13]. In CP, the all-different constraint has a dedicated filtering algorithm based on bipartite matching [22]. CPLEX CP Optimizer [15] is a CP engine with automatic search, while OR-Tools CP-SAT [19] combines CP propagators with a CDCL SAT core. Gurobi [10] and CPLEX [12] solve the ILP model by LP-based branch and cut.

The five AMO methods above are established constructions, and the course assignment asks for their implementation and comparison rather than a new encoding. The contribution of this report is to place all five encodings, four CDCL backends, and four CP/ILP solvers in one harness with identical problem semantics, independent solution validation, and a timing breakdown that separates process startup and license acquisition from model construction and search.

3. Problem definition and baseline models

3.1. Problem definition

Let $I = \{0, \dots, n-1\}$. A placement is a vector $q = (q_0, \dots, q_{n-1})$ where $q_r \in I$ is the column of the queen in row r. It is feasible iff

$$q_r \neq q_s \qquad \forall r < s, \tag{1}$$

$$q_r - r \neq q_s - s \qquad \forall r < s, \tag{2}$$

$$q_r + r \neq q_s + s \qquad \forall r < s. \tag{3}$$

There is no optimization objective: the first feasible placement suffices. All models below describe precisely these placements. Figure 1 shows a placement returned during the experiments. Rows and columns in the figure are labeled from one; source code and saved solutions use zero-based coordinates.

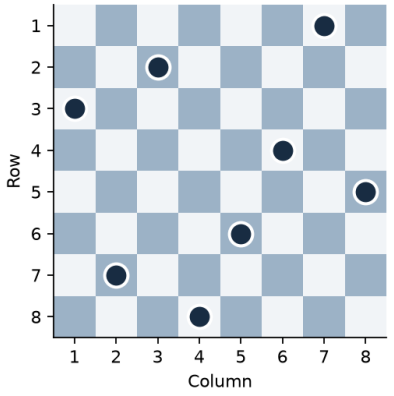


Figure 1: An independently validated solution of the 8×8 puzzle. Each dot denotes one queen; no two dots share a column or diagonal.

3.2. Constraint programming

The CP formulation introduces n integer variables $q_r \in I$ and the three global constraints

$$\text{AllDifferent}(q_0, \dots, q_{n-1}), \tag{4}$$

$$\text{AllDifferent}(q_0, q_1 + 1, \dots, q_{n-1} + n - 1), \tag{5}$$

$$\text{AllDifferent}(q_0, q_1 - 1, \dots, q_{n-1} - (n - 1)). \tag{6}$$

The same expressions are supplied to CPLEX CP Optimizer [15] through DOcplex [11] and to OR-Tools CP-SAT [19]. The counts n variables and three constraints refer to the user model; solver-internal expansion and presolve may add variables and constraints. In particular, CP-SAT is not a pure finite-domain CP engine: it combines SAT-based reasoning with other techniques. Its inclusion provides an exact baseline through a different modeling interface, rather than a disjoint category of algorithms.

3.3. Binary ILP formulation

Define $x_{rc} \in \{0, 1\}$ for each $(r, c) \in I^2$. Let $D_d^- = \{(r, c) : r - c = d\}$ and $D_s^+ = \{(r, c) : r + c = s\}$. The feasibility model is

$$\sum_{c \in I} x_{rc} = 1 \qquad \forall r \in I, \tag{7}$$

$$\sum_{r \in I} x_{rc} = 1 \qquad \forall c \in I, \tag{8}$$

$$\sum_{(r,c) \in D_d^-} x_{rc} \le 1 \qquad \forall d : |D_d^-| \ge 2, \tag{9}$$

$$\sum_{(r,c) \in D_s^+} x_{rc} \le 1 \qquad \forall s : |D_s^+| \ge 2. \tag{10}$$

The constant objective is zero. For $n \ge 2$ there are n^2 binary variables and $6n-6$ linear constraints. Singleton diagonals need no constraint. Gurobi [10] and CPLEX MIP [12] receive the identical model, with one thread, an explicit seed, and the default presolve settings. No model receives symmetry-breaking constraints, a warm start, or a precomputed placement.

4. SAT encoding

4.1. Board variables and formulations

The Boolean variable x_{rc} means that square (r, c) contains a queen and has DIMACS index $rn + c + 1$. For a literal set X, define

$$\mathrm{ALO}(X) = \bigvee_{x \in X} x, \qquad \mathrm{EO}(X) = \mathrm{ALO}(X) \wedge \mathrm{AMO}(X). \tag{11}$$

Write R_r, C_c, and $\mathcal{D}$ for row, column, and nonsingleton diagonal literal sets. The minimal formula is

$$F_{\min} = \bigwedge_r \mathrm{EO}(R_r) \ \wedge \ \bigwedge_c \mathrm{AMO}(C_c) \ \wedge \ \bigwedge_{D \in \mathcal{D}} \mathrm{AMO}(D). \tag{12}$$

The extended formula adds column at-least-one clauses:

$$F_{\mathrm{ext}} = F_{\min} \wedge \bigwedge_c \mathrm{ALO}(C_c). \tag{13}$$

Both formulas have exactly the same feasible board placements. Every model of $F_{\min}$ has exactly n queens by row EO. Column AMO distributes these queens among n columns with no repetition, so every column is occupied. Diagonal AMO then gives Equations (2)–(3). Conversely, every valid placement satisfies all constraints in both formulations, and the auxiliary variables of each correct AMO encoding can be extended to satisfy its CNF. The extended formula contains n additional clauses and no additional variables. Although logically redundant, those clauses may change propagation and search.

4.2. Five at-most-one encodings

Let $X = (x_1, \ldots, x_k)$ be an ordered sequence of distinct input variables. An encoding $E(X, Y)$ may introduce auxiliary variables Y, but must satisfy

$$\mathrm{AMO}(X) \quad \Longleftrightarrow \quad \exists Y \ E(X, Y). \tag{14}$$

In particular, an assignment with no true input must remain possible. All auxiliaries are fresh for each AMO group, including recursive calls. Groups of size zero or one need no clauses. The formulas below describe the implementation used to produce the measurements.

4.2.1. Pairwise

The direct encoding excludes every pair of true inputs:

$$E_{\mathrm{pair}}(X) = \bigwedge_{1 \le i < j \le k} (\neg x_i \vee \neg x_j). \tag{15}$$

Any two true inputs violate their pair clause; conversely, zero or one true input satisfies all pair clauses. There are $\binom{k}{2}$ clauses and no auxiliary variables. For $k = 3$ these are $(\neg x_1 \vee \neg x_2)$, $(\neg x_1 \vee \neg x_3)$, and $(\neg x_2 \vee \neg x_3)$. Setting x_2 true immediately forces x_1 and x_3 false.

4.2.2. Binary

The implementation calls PySAT's bitwise AMO encoder [21]. Let $b = \lceil \log_2 k \rceil$ and introduce $y_0, \ldots, y_{b-1}$. Assign input x_i the b-bit representation of $i - 1$, ordered from most to least significant bit. Write $\ell_{it} = y_t$ when bit position t in that order is one and $\ell_{it} = \neg y_t$ otherwise. The clauses are

$$E_{\mathrm{bin}}(X, Y) = \bigwedge_{i=1}^{k} \bigwedge_{t=0}^{b-1} (\neg x_i \vee \ell_{it}). \tag{16}$$

Two true inputs require different values of some bit, producing a contradiction. With one true input, choose its pattern; with none, the bits are unrestricted. For $k \ge 3$, the encoding uses b auxiliaries and kb clauses. PySAT simplifies $k = 2$ to one pairwise clause with no bits. For four inputs with patterns $00, 01, 10, 11$, selecting x_3 fixes pattern 10. Each other input disagrees with a fixed bit and is propagated false.

4.2.3. Commander

The grouping construction follows Klieber and Kwon [14]. For $k > 4$, partition X into consecutive groups $G_1, \ldots, G_m$ of at most three inputs, where $m = \lceil k/3 \rceil$. Introduce commander a_j for each group and use

$$E_{\text{cmd}}(X, A) = \bigwedge_j E_{\text{pair}}(G_j) \wedge \bigwedge_j \bigwedge_{x \in G_j} (\neg x \vee a_j) \wedge \bigwedge_j \left(\neg a_j \vee \bigvee_{x \in G_j} x \right) \wedge E_{\text{cmd}}(A). \tag{17}$$

Recursion ends at $k \leq 4$ with pairwise clauses. The within-group clauses exclude two selections in one group; forward implications and commander AMO exclude selections in different groups. Assigning each commander the disjunction of its members extends every valid input assignment, including the all-false assignment. Reverse implications are included in this implementation, although removing them preserves the projected AMO relation. This is a stated implementation choice, not a claim that every commander encoder uses the same clauses or cutoff.

4.2.4. Recursive product

This implementation uses the recursive two-dimensional product construction [4]. For $k > 4$, let $p = \lceil \sqrt{k} \rceil$ and $q = \lceil k/p \rceil$. Introduce $U = (u_1, \ldots, u_p)$ and $V = (v_1, \ldots, v_q)$. Input x_i occupies $r(i) = \lfloor (i-1)/q \rfloor + 1$, $c(i) = ((i-1) \bmod q) + 1$. Add

$$E_{\text{prod}} = \bigwedge_{i=1}^{k} \left[(\neg x_i \vee u_{r(i)}) \wedge (\neg x_i \vee v_{c(i)}) \right] \wedge E_{\text{prod}}(U) \wedge E_{\text{prod}}(V). \tag{18}$$

The recursive AMO calls use pairwise clauses for at most four inputs. Distinct selected inputs differ in a row or column coordinate and therefore violate one coordinate AMO. A single selected input can be extended by choosing its two coordinates; for no selected inputs, choose all coordinates false. Unused grid cells need no extra exclusion clauses for this projected AMO relation, and this implementation adds none. For $k = 6$, a 3×2 grid maps x_1, x_2 to the first grid row, x_3, x_4 to the second, and x_5, x_6 to the third.

4.2.5. Sequential counter

PySAT supplies the AMO specialization of Sinz's sequential counter [23]. For $k \geq 3$, introduce $s_1, \ldots, s_{k-1}$ and add

$$(\neg x_1 \vee s_1), \qquad (\neg x_k \vee \neg s_{k-1}), \qquad \bigwedge_{i=2}^{k-1} \left[(\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg x_i \vee \neg s_{i-1}) \right]. \tag{19}$$

A true input forces a true prefix indicator that propagates forward, preventing any later input from being true. A satisfying extension sets $s_i = \bigvee_{j \leq i} x_j$; the clauses do not require this equality for every auxiliary assignment. The size is $k - 1$ auxiliaries and $3k - 4$ clauses. PySAT reduces $k = 2$ to one pairwise clause without auxiliaries.

4.2.6. Propagation on the original inputs

Each construction above has the following useful property for an isolated AMO: selecting one input makes unit propagation falsify every other input. Pairwise uses the pair clauses directly. Binary fixes a unique bit pattern. Commander propagates to the selected group's commander, excludes other commanders recursively, and then excludes their members. Product selects one row and column coordinate and excludes every other input through at least one coordinate. Sequential propagates true prefix indicators forward and false prefix indicators backward.

For example, with four sequential inputs, setting x_3 true forces s_3 true and s_2 false. The latter forces s_1 and x_2 false, then x_1 false; s_3 true forces x_4 false. This is an input-propagation property of one AMO, not complete propagation of the entire N-queens CNF. It also does not imply equal propagation cost or equal search time across encodings. The proof descriptions follow the displayed clauses; the additional automated checks are described below.

4.3. Formula-size analysis

Let $a_E(k)$ and $c_E(k)$ count generated auxiliary variables and clauses for one k-input AMO under encoding E. Both counts are zero for $k \le 1$. The direct counts are

$$(a_{\text{pair}}, c_{\text{pair}}) = (0, \tbinom{k}{2}), \tag{20}$$

$$(a_{\text{bin}}, c_{\text{bin}}) = (\lceil \log_2 k \rceil, k\lceil \log_2 k \rceil), \qquad k \ge 3, \tag{21}$$

$$(a_{\text{seq}}, c_{\text{seq}}) = (k-1, 3k-4), \qquad k \ge 3. \tag{22}$$

For both binary and sequential $k = 2$ the counts are $(0, 1)$. For both commander and product, $k \le 4$ uses the pairwise counts. For larger commander groups, with $t_j = |G_j|$ and $m = \lceil k/3 \rceil$, the implementation gives

$$a_{\text{cmd}}(k) = m + a_{\text{cmd}}(m), \tag{23}$$

$$c_{\text{cmd}}(k) = \sum_{j=1}^{m} \tbinom{t_j}{2} + k + m + c_{\text{cmd}}(m). \tag{24}$$

The terms count within-group pairs, k forward clauses, m reverse clauses, and recursive commander AMO. For product, with p, q as above,

$$a_{\text{prod}}(k) = p + q + a_{\text{prod}}(p) + a_{\text{prod}}(q), \tag{25}$$

$$c_{\text{prod}}(k) = 2k + c_{\text{prod}}(p) + c_{\text{prod}}(q). \tag{26}$$

These are counts for the actual recursive implementation; in particular, no unused-cell clauses enter the product recurrence.

For $n \ge 2$, there are $2n$ row and column AMO groups of length n. Across the two diagonal directions, there are two more groups of length n and four of each length $2, \dots, n-1$. Thus the total variables and minimal-formulation clauses are

$$V_E(n) = n^2 + (2n+2)a_E(n) + 4\sum_{k=2}^{n-1} a_E(k), \tag{27}$$

$$C_{E,\min}(n) = n + (2n+2)c_E(n) + 4\sum_{k=2}^{n-1} c_E(k). \tag{28}$$

The initial n clauses are row ALO. Extended adds exactly n column ALO clauses, giving $C_{E,\text{ext}} = C_{E,\min} + n$ and the same V_E. For $n = 1$, the counts are one variable and one minimal clause; extended generates a second, duplicate ALO clause. Counts refer to emitted clauses, before any solver simplification.

Table 1
Asymptotic size of the complete board CNF, derived from the implemented AMO constructions. Both formulations have these orders.

Encoding	Auxiliary variables	Clauses
Pairwise	0	$\Theta(n^3)$
Binary	$O(n \log n)$	$O(n^2 \log n)$
Commander	$O(n^2)$	$O(n^2)$
Recursive product	$O(n^{3/2})$	$O(n^2)$
Sequential counter	$O(n^2)$	$O(n^2)$

For pairwise, substituting $c(k) = \binom{k}{2}$ yields

$$C_{\text{pair},\min}(n) = \frac{5n^3 - 6n^2 + 4n}{3}, \qquad n \ge 2. \tag{29}$$

As a concrete check, a minimal 4×4 board has 16 variables and 80 clauses: four row ALO clauses, 48 row/column pair clauses, and 28 diagonal pair clauses. Extended has 84 clauses. Figure 2 plots the exact counts for the six encoding-study sizes. Counts are computed even when a timed run was unfinished.

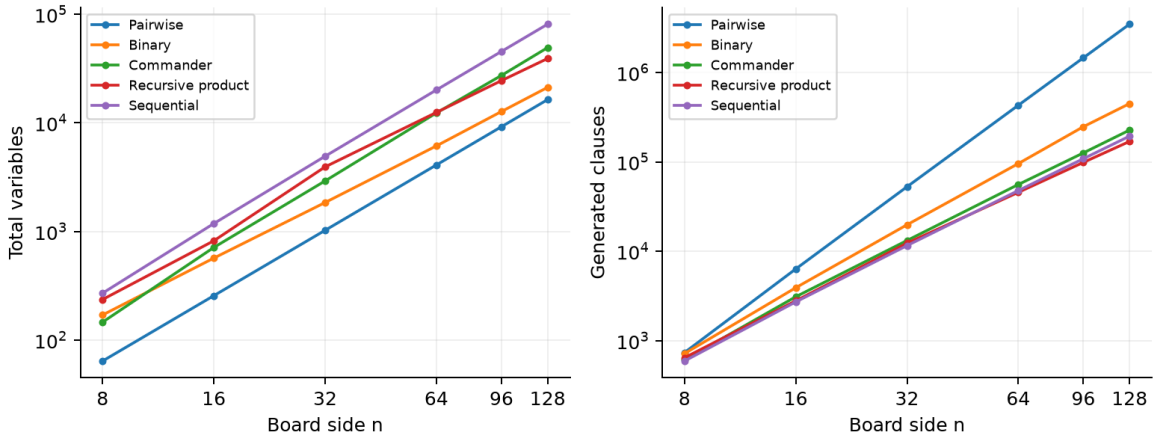


Figure 2: Exact generated CNF size for the minimal formulation. Both axes are logarithmic; total variables include board variables and auxiliaries. The curves describe construction size, not measured runtime.

The recursive product and commander constructions reduce clause growth, but their auxiliaries and propagation paths differ. The experiments therefore compare size with measured construction and search costs rather than treating the smallest CNF as the fastest solver configuration.

4.4. Implementation and correctness checks

The SAT encoder streams clauses into the chosen solver instead of retaining a second full Python copy of the formula. An export command produces standard DIMACS CNF for independent use. Decoding inspects only the first n^2 placement variables, so auxiliary assignments cannot create queens. Every SAT or CP/ILP solution passes a separate checker for length, domain, column uniqueness, and uniqueness of both diagonal indices.

The semantic test suite enumerates every truth assignment for AMO groups of size zero through eleven, for both positive and negative literals. It then enumerates all projected N-queens solutions for $n = 1, \ldots, 8$ for all ten SAT configurations. Blocking clauses use only selected board squares, so distinct auxiliary assignments cannot be counted as distinct placements. The resulting solution sets are compared with an independent backtracking implementation, yielding counts $1, 0, 0, 2, 10, 4, 40, 92$. Integration checks also confirm infeasibility at $n = 3$ and a valid solution at $n = 8$ for every CP/ILP backend. These tests verify semantic equivalence rather than only checking that the code runs. A separate unit-propagation test covers every selected input for group sizes one through 64, with both literal polarities; it does not invoke a SAT search. The formula-size recurrences above are also checked against the generated CNF before drawing the size figure.

5. Computational experiments

5.1. Environment and protocol

All runs were performed on Windows 11 (build 26200), with an AMD Ryzen 7 8845HS processor (16 logical processors), Python 3.14.5, PySAT 1.9.dev2, Gurobi 13.0.3, CPLEX Python package 22.2.0.1, DOcplex 2.32.264, and OR-Tools 9.15.6755. CPLEX CP uses the CP Optimizer executable from Optimization Studio 22.2. The saved configuration files record these versions and the complete command parameters. Gurobi uses a working WLS license; CPLEX MIP was verified to accept models beyond the community-edition size limit.

Each experiment runs sequentially in fresh child processes, with one solver thread and three repetitions. CP/ILP runs use seeds one through three. SAT backends retain their default parameters; their repetitions primarily measure timing variation and are not different randomized SAT seeds. Method order is rotated across repetitions. Every run has a hard ten-second deadline for the child process, including imports, license acquisition, construction, and solving. Exceeding this budget yields TO; it never implies infeasibility. The Windows harness also terminates child CP Optimizer processes on timeout. Recorded timeout duration includes termination and cleanup; operating-system delays can make this exceed the requested budget. Such records remain unfinished runs, regardless of the cleanup duration.

Two timing measures are kept: process time, measured by the parent, and core time, equal to model construction plus solver execution. Gurobi WLS startup is recorded separately as setup time; it is included in process time but excluded from core time. The main comparison uses process time to reflect the budget actually enforced. Both timings are reported to avoid attributing all license and import overhead to search performance. Construction and solve times cannot be recovered when the process is killed before returning; these fields remain empty rather than fabricated.

For successful configurations, per-size tables show the median of three process times. A cell TO ($s/3$) means only s of three runs finished. Aggregate rankings use PAR-2: a successful run contributes its process time, and an unfinished or erroneous run contributes twice the ten-second budget. This prevents fast successful subsets from hiding failures. The benchmark contains one standard board per size; repetitions are repeated measurements, not distinct instances or independent evidence of broad statistical significance.

5.2. Encoding and formulation comparison

The first experiment fixes CaDiCaL 1.9.5 and evaluates all five AMO encodings under both formulations at $n \in \{8, 16, 32, 64, 96, 128\}$. It contains $6 \times 10 \times 3 = 180$ runs. Table 2 includes exact variable and clause counts at $n = 128$, recomputed from the encoder independently of whether a particular timed run finished. Figure 3 shows PAR-2 by board size, and Table 7 in Appendix A lists the median process time of every configuration at every size.

Table 2
Encoding comparison over 18 runs per configuration. Structural counts refer to the 128×128 board. Lower PAR-2 is better.

Encoding / formulation	Solved	PAR-2 (s)	Variables	Clauses
commander/minimal	18/18	0.450	49,354	226,242
commander/extended	18/18	0.459	49,354	226,370
product/extended	18/18	0.921	39,174	169,112
binary/extended	18/18	1.221	21,234	447,828
seqcounter/extended	18/18	1.275	81,150	193,800
pairwise/minimal	18/18	1.863	16,384	3,462,656
pairwise/extended	18/18	1.898	16,384	3,462,784
binary/minimal	18/18	3.113	21,234	447,700
seqcounter/minimal	15/18	5.718	81,150	193,672
product/minimal	12/18	7.534	39,174	168,984

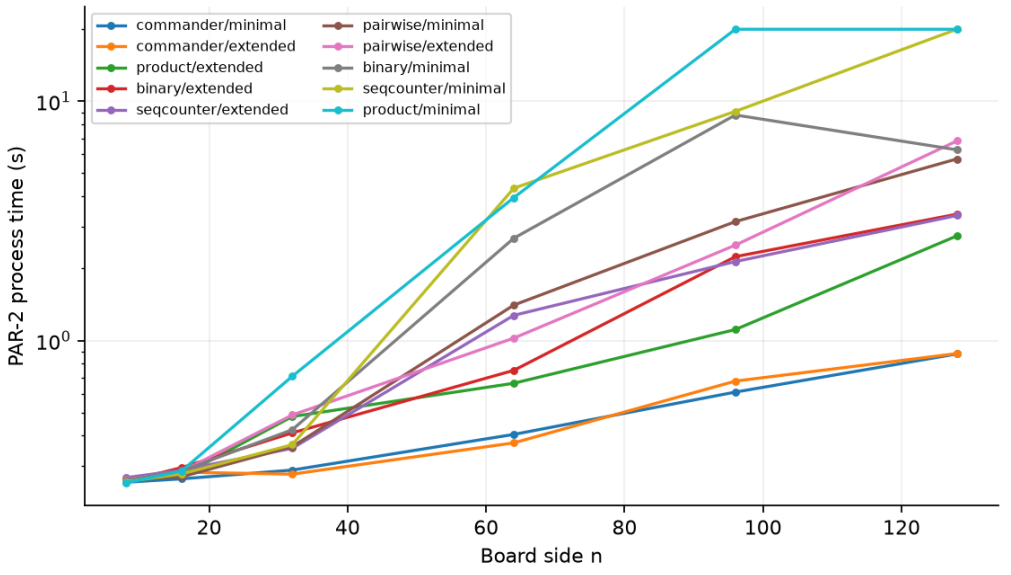


Figure 3: Encoding and formulation comparison with CaDiCaL. Timeouts contribute 20 seconds each to PAR-2.

The lowest aggregate PAR-2 is obtained by commander/minimal, at 0.450 seconds, with 18 of 18 runs solved. At $n = 128$, pairwise/minimal has 3,462,656 clauses, whereas commander/minimal has 226,242. The ranking is measured rather than inferred from clause count alone. The reference used in the following experiments is commander/minimal; it was selected after an exploratory encoding comparison on the same size family. The reported encoding results were then collected with the finalized harness. For product, the minimal formulation solves 12/18 runs with PAR-2 7.534 seconds; adding column ALO solves 18/18 with PAR-2 0.921 seconds. For seqcounter, the minimal formulation solves 15/18 runs with PAR-2 5.718 seconds; adding column ALO solves 18/18 with PAR-2 1.275 seconds. Thus the added column clauses can improve measured runtime without changing the set of placements, although the effect depends on the encoding.

Construction cost and repeated-run variation. Table 3 separates construction and search at $n = 128$. Build time includes generating clauses and inserting them into the SAT solver; solve time measures the solver call. The process column gives a median and observed minimum–maximum range across the three repetitions. Each timing column is summarized separately, so its medians need not add up. A range from three repeats is descriptive, not a confidence interval.

Table 3
Encoding-study times at $n = 128$, in seconds. Completed cells contain three valid solutions. No component times are imputed for timeouts.

Encoding / formulation	Build	Solve	Process [min, max]
pairwise/minimal	4.050	1.332	5.795 [5.474, 5.954]
pairwise/extended	3.901	2.593	6.769 [6.749, 6.963]
binary/minimal	0.566	5.385	6.218 [6.179, 6.407]
binary/extended	0.578	2.514	3.362 [3.111, 3.646]
commander/minimal	0.325	0.240	0.886 [0.858, 0.892]
commander/extended	0.327	0.257	0.888 [0.867, 0.890]
product/minimal	–	–	TO (0/3)
product/extended	0.247	2.220	2.736 [2.644, 2.830]
seqcounter/minimal	–	–	TO (0/3)
seqcounter/extended	0.349	2.693	3.331 [3.185, 3.472]

For pairwise/minimal, the median construction time is 4.050 s, compared with 0.325 s for commander/minimal. The ratio is approximately 12.5 on this implementation. This connects the clause-count reduction to an observed construction saving. It does not determine search performance: product/extended has fewer clauses than commander/minimal but a longer solver call.

Effect of redundant column clauses. At $n = 128$, product/minimal and seqcounter/minimal each finish zero of three runs, while their extended versions finish all three with median process times 2.736 s and 3.331 s. In each case, extended adds only 128 column ALO clauses and no variables. These explicit column disjunctions make the column occupancy requirement directly available to unit propagation; in minimal it follows from the combined row and column constraints. The measured improvement is consistent with a change in propagation or search, but timings alone do not isolate which internal events caused it. Commander receives the same additional clauses without a comparable improvement. The benefit is therefore conditional on the encoding, not a universal advantage of adding redundant clauses.

5.3. SAT solver comparison

The second experiment fixes the selected reference encoding and compares CaDiCaL 1.9.5 [3], Glucose 4.2 [1], MiniSat 2.2 [5], and MapleChrono [17] through PySAT. These backends were checked locally before benchmarking. The same six sizes and three repetitions give $6 \times 4 \times 3 = 72$ runs. The selected encoding is disclosed below; choosing it on the same board family is an in-sample engineering choice rather than a held-out prediction.

Table 4
Median process time (seconds) for the four SAT backends with a fixed reference encoding. TO includes the number of completed runs.

n	cadical195	glucose42	minisat22	maplechrono
8	0.253	0.256	0.260	0.255
16	0.257	0.293	0.273	5.354
32	0.278	7.891	0.286	6.866
64	0.359	4.073	0.364	TO (0/3)
96	0.637	TO (0/3)	0.513	TO (0/3)
128	0.846	TO (0/3)	1.956	TO (0/3)

All backends use commander/minimal. Their aggregate PAR-2 values are cadical195: 0.441 seconds, minisat22: 0.609 seconds, glucose42: 8.755 seconds, maplechrono: 12.081 seconds. These results describe this fixed formula family and short budget; they do not establish a universal ranking of SAT engines.

At $n = 32$, CaDiCaL's median process time is 0.278 s with range [0.267, 0.281], whereas Glucose records 7.891 s with range [7.885, 7.948]. The difference exceeds the observed timing spread, although three repetitions do not establish a statistical ranking. At $n = 96$, MiniSat is faster than CaDiCaL in the median table, while CaDiCaL is faster at $n = 128$. The aggregate winner need not win at every size.

Because the formula is fixed, these differences cannot be explained by choosing a different AMO construction for each backend. The saved SAT records include aggregate search counters, but no controlled ablation of branching or learned-clause policies was conducted. The report therefore does not assign the runtime differences to a specific solver heuristic.

5.4. SAT versus CP and ILP

The third experiment compares the reference SAT configuration with CPLEX CP, CPLEX MIP, Gurobi, and CP-SAT. It uses the six original sizes plus $n = 256$ and $n = 512$, retaining the same three repetitions, thread count, and ten-second process deadline. There are $8 \times 5 \times 3 = 120$ runs in the combined main and scaling experiments.

Table 5
Median process time (seconds) for exact methods. Every fully solved cell represents three validated placements.

n	SAT	CPLEX CP	CPLEX MIP	Gurobi	CP-SAT
8	0.247	1.510	0.401	0.345	1.325
16	0.264	1.496	0.427	0.357	1.354
32	0.272	1.443	0.427	0.344	1.370
64	0.363	1.466	0.467	0.423	1.725
96	0.685	1.462	0.517	0.412	2.529
128	0.871	1.503	0.606	0.700	3.838
256	4.085	1.419	1.248	0.854	TO (0/3)
512	TO (0/3)	1.440	5.617	2.367	TO (0/3)

Table 6
Median construction-plus-solve time (seconds) for the same runs. Imports and Gurobi WLS acquisition are excluded; deadline status is unchanged.

n	SAT	CPLEX CP	CPLEX MIP	Gurobi	CP-SAT
8	0.002	0.040	0.005	0.004	0.011
16	0.005	0.039	0.009	0.005	0.027
32	0.024	0.039	0.017	0.010	0.089
64	0.101	0.041	0.053	0.079	0.443
96	0.364	0.046	0.108	0.056	1.223
128	0.569	0.049	0.194	0.338	2.538
256	3.610	0.061	0.848	0.403	TO (0/3)
512	TO (0/3)	0.092	5.090	1.530	TO (0/3)

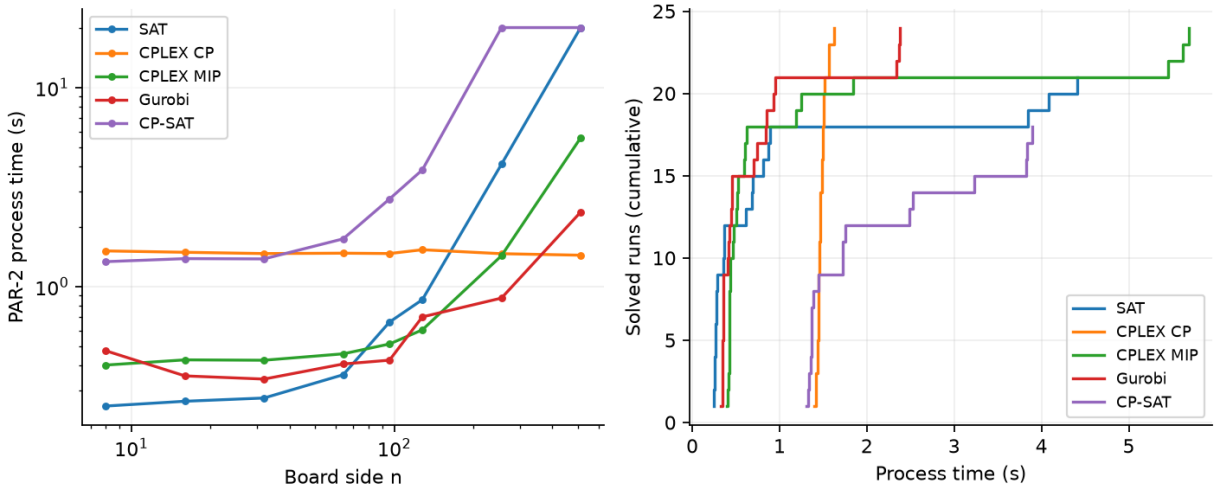


Figure 4: Exact-method comparison: PAR-2 versus board size (left) and cumulative solved runs versus process time (right). The cumulative plot counts repetitions rather than distinct sizes.

Across the 24 runs per method, SAT solves 21/24 with PAR-2 3.348 seconds; CPLEX CP solves 24/24 with PAR-2 1.477 seconds; CPLEX MIP solves 24/24 with PAR-2 1.230 seconds; Gurobi solves 24/24 with PAR-2 0.743 seconds; CP-SAT solves 18/24 with PAR-2 6.553 seconds. Timeout cells in Tables 5 and 6 preserve unsuccessful runs. On small boards, process times include library and license startup that may exceed actual search time. At larger sizes, the core-time table shows differences in construction and search costs, while startup remains part of the process deadline. The input-model size alone does not explain those costs. At $n = 512$, SAT completes 0/3 runs; CPLEX CP completes

3/3 runs; CPLEX MIP completes 3/3 runs; Gurobi completes 3/3 runs; CP-SAT completes 0/3 runs. The CP user model at this size has 512 integer variables and three global constraints; the ILP user model has 262,144 binary variables and 3,066 linear constraints. These are input-model counts, not the numbers remaining after presolve.

The choice of timing metric changes the small-board interpretation. At $n = 8$, reference SAT has median process time 0.247 s, compared with 1.510 s for CPLEX CP. Their construction-plus-solve medians are only 0.002 s and 0.040 s. Much of the end-to-end difference at this size lies outside the measured core interval. Process time remains the primary metric because the deadline includes those startup costs.

At $n = 512$, CPLEX CP, CPLEX MIP, and Gurobi have median core times 0.092, 5.090, and 1.530 s, respectively. CPLEX CP is fastest for this core metric and also for process time at this particular size, whereas Gurobi has the lowest aggregate PAR-2 across the eight sizes. These are two different observations. The small CP input model alone is insufficient to prove why its engine performs well; solver preprocessing and search were not isolated experimentally.

5.5. Limitations

The ten-second deadline measures short-budget behavior, so a TO is not evidence that a method cannot solve that size with more time. Core times should not be read as successful runs under a separate ten-second search budget: all outcomes were determined by the same process deadline. Startup overhead is material on small boards, especially for CP-SAT imports and Gurobi WLS acquisition. Hardware power management and background tasks may introduce timing noise despite sequential execution.

Only unconstrained square boards and default solver settings are studied. The selected SAT encoding is optimized using the same family later used for comparison, and the report does not claim an unbiased general ranking of solver technology. In particular, better CP, ILP, or SAT formulations, specialized constructions, symmetry breaking, or different parameter settings could change the ordering. Large-board tests concern finding one placement, not exhaustive counting. Completion instances would require additional benchmarks and conclusions.

6. Conclusion

The delivered implementation solves standard N-queens through SAT and four required CP/ILP baselines, exports CNF, independently checks placements, and reproduces the measured comparisons. The encoding study separates formula size from observed runtime, while the formulation study isolates the effect of redundant column clauses. Process and core timings make startup costs explicit. Complete small-board solution-set tests support the correctness of all ten SAT configurations, and saved placements permit independent validation of every successful benchmark run. The final audit covers 372 timed runs and independently revalidates 339 returned placements. There are 33 timeouts and 0 solver errors. The measured differences support selecting a suitable encoding and modeling interface for the available budget; they do not imply that any generic solver replaces specialized N-queens constructions.

Possible extensions

A small follow-up could vary one implementation choice at a time: commander reverse implications, the product cutoff, or a valid symmetry-breaking constraint. Any symmetry restriction should be applied consistently to all compared models, and its effect on finding one placement should be distinguished from enumerating all placements. A longer deadline would also show whether the present timeout boundary persists beyond the short-budget setting. Finally, completion instances with fixed queens, including satisfiable and infeasible boards, would provide a different test family. These are proposed extensions; none is part of the measurements reported here.

CRediT authorship contribution statement

Do Duc Long: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Source code, tests, raw CSV results, JSONL placements, configuration files, and report sources are available at https://github.com/dolong226/homework1-nqueens. Gurobi license credentials are local configuration and are excluded from the repository.

A. Encoding results by board size

Table 7
Median process time (seconds) of each encoding and formulation with CaDiCaL, over three repetitions per cell. TO (s/3) means only s of three runs finished within the ten-second deadline.

Encoding / formulation	$n = 8$	$n = 16$	$n = 32$	$n = 64$	$n = 96$	$n = 128$
pairwise/minimal	0.253	0.271	0.355	1.262	3.141	5.795
binary/minimal	0.263	0.286	0.413	2.667	8.791	6.218
commander/minimal	0.253	0.261	0.282	0.382	0.609	0.886
product/minimal	0.254	0.285	0.708	3.738	TO (0/3)	TO (0/3)
seqcounter/minimal	0.260	0.265	0.348	4.294	8.936	TO (0/3)
pairwise/extended	0.268	0.272	0.484	0.987	2.516	6.769
binary/extended	0.259	0.289	0.405	0.751	2.217	3.362
commander/extended	0.263	0.262	0.276	0.376	0.666	0.888
product/extended	0.266	0.269	0.481	0.648	1.105	2.736
seqcounter/extended	0.252	0.278	0.355	1.250	2.128	3.331

Declaration of generative AI and AI-assisted technologies

OpenAI Codex assisted with implementation, experiment automation, and report drafting. Claude Code (Anthropic) assisted with the final revision of the report (related work, references, and appendix) and of the reproduction scripts. The automated correctness and solution-validation checks are documented above, and the raw measurements and placements are supplied for inspection. These checks cover the tested cases and do not replace the author's responsibility for the submitted work.

References

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI 2009), pp. 399–404.

[2] Bell, J., Stevens, B., 2009. A survey of known results and research areas for n-queens. Discrete Mathematics 309, 1–31. doi:10.1016/j.disc.2007.12.043.

[3] Biere, A., Fazekas, K., Fleury, M., Heisinger, M., 2020. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT competition 2020, in: Proceedings of SAT Competition 2020 – Solver and Benchmark Descriptions, University of Helsinki. pp. 51–53.

[4] Chen, J., 2010. A new SAT encoding of the at-most-one constraint. Workshop on Constraint Modelling and Reformulation (ModRef).

[5] Eén, N., Sörensson, N., 2004. An extensible SAT-solver, in: Theory and Applications of Satisfiability Testing – SAT 2003, pp. 502–518. doi:10.1007/978-3-540-24605-3_37.

[6] Frisch, A.M., Giannaros, P.A., 2010. SAT encodings of the at-most-k constraint: Some old, some new, some fast, some slow. Workshop on Constraint Modelling and Reformulation (ModRef).

[7] Frisch, A.M., Peugniez, T.J., 2001. Solving non-Boolean satisfiability problems with stochastic local search, in: Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI 2001), pp. 282–288.

[8] Gent, I.P., Jefferson, C., Nightingale, P., 2017. Complexity of n-queens completion. Journal of Artificial Intelligence Research 59, 815–848. doi:10.1613/jair.5512.

[9] Google, n.d. The N-queens problem: OR-Tools. https://developers.google.com/optimization/cp/queens. Accessed 10 October 2026.

[10] Gurobi Optimization, n.d. Gurobi optimizer reference manual. https://docs.gurobi.com/projects/optimizer/en/current/. Version 13.0.3 used in the experiments.

[11] IBM, n.d.a. DOcplex constraint programming model documentation. https://ibmdecisionoptimization.github.io/docplex-doc/cp/docplex.cp.model.py.html. Accessed 10 October 2026.

[12] IBM, n.d.b. IBM ILOG CPLEX optimization studio. https://www.ibm.com/products/ilog-cplex-optimization-studio. Version 22.2 used in the experiments.

[13] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, pp. 428–437. doi:10.1007/978-3-319-94144-8_26.

[14] Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects. Workshop on Constraints in Formal Verification. URL: https://www.cs.cmu.edu/~wklieber/papers/2007_efficient-cnf-encoding-for-selecting-1.pdf.

[15] Laborie, P., Rogerie, J., Shaw, P., Vilím, P., 2018. IBM ILOG CP optimizer for scheduling. Constraints 23, 210–250. doi:10.1007/s10601-018-9281-x.

[16] Marques-Silva, J.P., Sakallah, K.A., 1999. GRASP: A search algorithm for propositional satisfiability. IEEE Transactions on Computers 48, 506–521. doi:10.1109/12.769433.

[17] Nadel, A., Ryvchin, V., 2018. Chronological backtracking, in: Theory and Applications of Satisfiability Testing – SAT 2018, pp. 111–121. doi:10.1007/978-3-319-94144-8_7.

[18] Nguyen, V.H., Mai, S.T., 2015. A new method to encode the at-most-one constraint into SAT, in: Proceedings of the Sixth International Symposium on Information and Communication Technology (SoICT 2015), pp. 46–53.

[19] Perron, L., Didier, F., n.d. CP-SAT. Google OR-Tools, https://developers.google.com/optimization/cp/cp_solver/. Version 9.15 used in the experiments.

[20] Prestwich, S., 2021. CNF encodings, in: Biere, A., Heule, M., van Maaren, H., Walsh, T. (Eds.), Handbook of Satisfiability. 2 ed.. IOS Press, pp. 75–100.

[21] PySAT Developers, n.d. Cardinality encodings: pysat.card. https://pysathq.github.io/docs/html/api/card.html. Accessed 10 October 2026.

[22] Régin, J.C., 1994. A filtering algorithm for constraints of difference in CSPs, in: Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI-94), pp. 362–367.

[23] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, pp. 827–831. doi:10.1007/11564751_73.